

# AUTOBAHN

Arhitektura za BFT konsenzus bez zastoja

# Plan predavanja

- |   |                              |                                                                               |
|---|------------------------------|-------------------------------------------------------------------------------|
| 1 | <b>Problem</b>               | Autobahn je arhitektura, Blip, Hangover                                       |
| 2 | <b>CAR i arhitektura</b>     | Osnovna jedinica podataka i podela na dva nezavisna sloja                     |
| 3 | <b>Faza razmene podataka</b> | Kako validatori kontinuirano prikupljaju i overavaju podatke — sa pseudokodom |
| 4 | <b>Faza konsenzusa</b>       | Kako se glasa o redosledu — Fast/Slow Path i View Change, sa pseudokodom      |
| 5 | <b>Zaključak</b>             | Šta pokazuju rezultati                                                        |

# Zašto je BFT konsenzus spor?

U distribuiranom sistemu sa  $n = 3f+1$  čvorova, do  $f$  može biti neispravno ili zlonamerno. Svi ispravni čvorovi ipak moraju da se slože oko identičnog redosleda transakcija.

## BLIP

Period bez napretka — mrežno kašnjenje, pad čvora, ili zlonamerni lider zaustavljaju protokol.

## HANGOVER

Performanse ostaju loše i posle oporavka mreže — protokolu treba dugo da se vrati na punu brzinu.

# Dve škole mišljenja — Autobahn-ov spoj

## Leader-based

*npr. Tendermint, HotStuff*

- Lider šalje ceo blok na glasanje
- Jednostavno, ali podaci putuju TOKOM glasanja
- Spor oporavak posle pada lidera

## DAG-based

*npr. Bullshark, Narwhal*

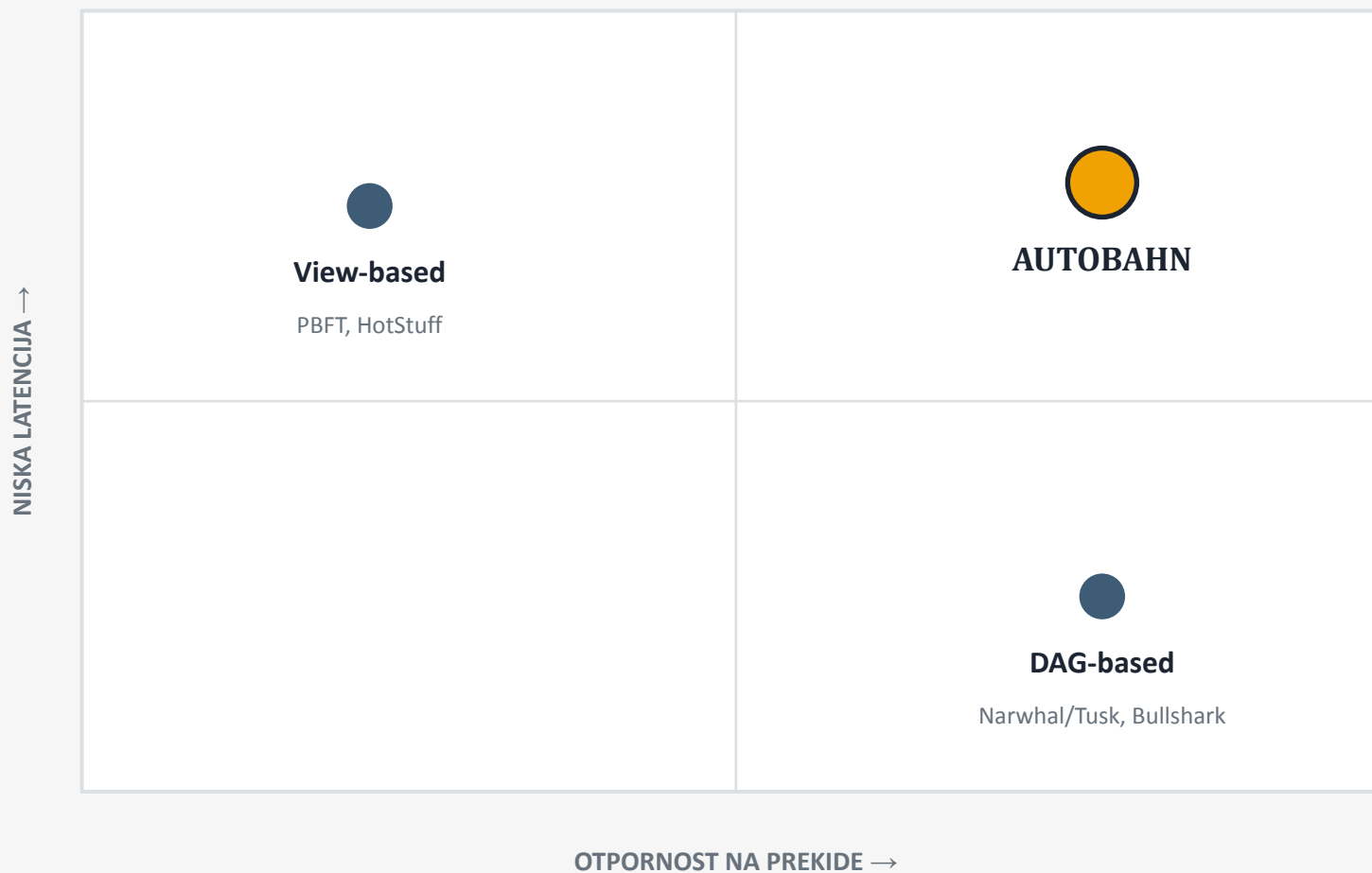
- Podaci se šire kroz DAG unapred
- Bolji throughput, ali glasa se o samim blokovima
- Konsenzus i dalje nosi teret podataka

## Autobahn

*hibridni pristup*

- Diseminacija i konsenzus su POTPUNO odvojeni
- Glasa se samo o hash-evima (TipCut), nikad o transakcijama
- Cilj: brzina DAG-a + niska latencija leader-based sistema

# Kompromis koji niko nije rešio — do sada



# Dva nezavisna sloja

## Sloj diseminacije podataka

*Asinhron, radi kontinuirano u pozadini. Zanima ga samo da su podaci dostupni.*

- Pravi CAR-ove (batch + hash + lanac)
- Širi ih gossip protokolom
- Prikuplja parcijalne potpise
- Formira Proof of Availability (PoA)

## Sloj konsenzusa

*Parcijalno sinhron, počinje TEK kad postoji PoA. Zanima ga samo redosled.*

- Lider gradi TipCut od CAR-ova sa PoA
- Validatori glasaju (Fast/Slow Path)
- Formira se Quorum Certificate
- Redosled se finalizuje i izvršava

# Autobahn je arhitektura, a ne algoritam

Bilo koji BFT protokol za glasanje može se „motorizovati“ tako što se ispod njega postavi Autobahnov sloj diseminacije — umesto glasanja o pojedinačnim blokovima, glasa se o TipCut-ovima.

1

## Implementiraj sloj diseminacije

Svaka linija neprestano objavljuje CAR-ove;  $f+1$  potpisa formira PoA — dokaz dostupnosti, ne i ispravnosti sadržaja.

2

## Prilagodi postojeći BFT protokol

Umesto celih blokova, protokol prima i glasa o TipCut-ovima — malim paketima referenci na CAR-ove sa PoA.

3

## Sortiraj i izvrši

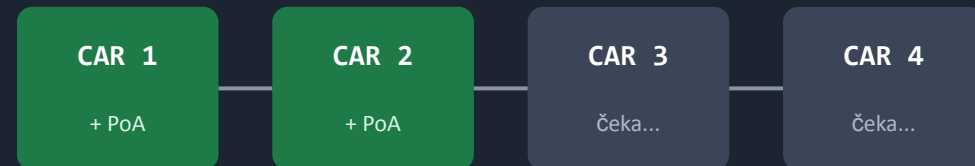
Kada se TipCut finalizuje, svi CAR-ovi u njemu se deterministički sortiraju i transakcije izvršavaju tim redosledom.

# CAR ( Certification of Available Request)

## struct CAR

|            |                                       |
|------------|---------------------------------------|
| id         | redni broj u traci                    |
| lane_id    | identifikator validatora / trake      |
| prev_hash  | hash prethodnog CAR-a u ISTOJ traci   |
| batch_hash | hash celog batch-a transakcija        |
| PoA        | dokaz dostupnosti (dodaje se kasnije) |

## Lanac unutar jedne trake:





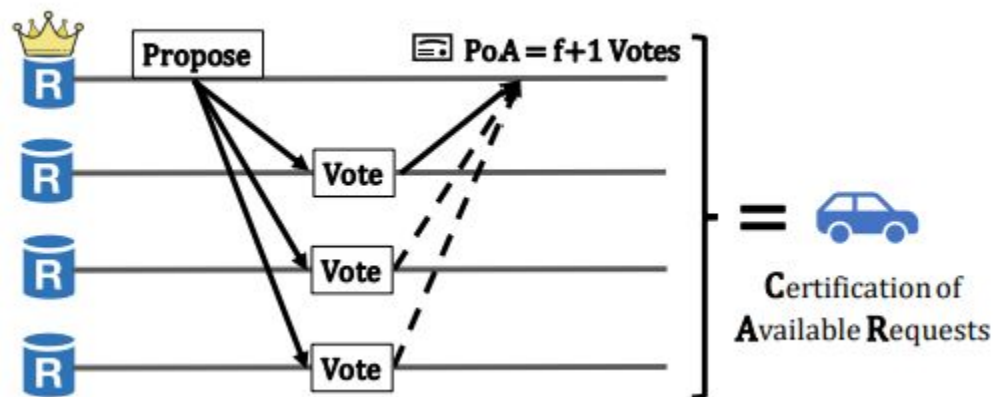
# Kako nastaje jedan CAR

## AUTOR CAR-A

- 1 **Sakupi batch** Skuplja transakcije iz mempool-a do vremenskog prozora (npr. 50ms) ili limita veličine (npr. 1MB).
- 2 **Hešuje i lanči** Računa batch\_hash i dodaje prev\_hash — hash svog prethodnog CAR-a u traci.
- 3 **Potpisuje i emituje** Potpisuje agregat privatnim ključem i šalje (gossip) CAR svim validatorima.

## OSTALI VALIDATORI

- 1 **Proverava strukturu** Da li je potpis validan i da li se prev\_hash poklapa sa lokalnom kopijom trake.
- 2 **Proverava samo sintaksu tx** NE proverava stanje (npr. balans) — samo format i potpise transakcija.
- 3 **Čuva i šalje parcijalni potpis** Upisuje batch na disk, šalje mali potpis (vote share) nazad autoru.



# Proof of Availability — prag $f+1$

*$N = 3f+1$  validatora, najviše  $f$  zlonamerno  $\rightarrow$  ako sakupimo  $f+1$  potpisa, bar JEDAN potpisnik je sigurno ispravan.*

1

**Zašto je to dovoljno**

Zlonamernih ima najviše  $f$ , a mi imamo  $f+1$  potpisa — matematički nemoguće da su svi zlonamerni.

2

**Šta PoA garantuje**

Bar jedan pošten validator ima podatke i može ih deliti na zahtev, čak i ako autor CAR-a nestane.

3

**BLS agregacija potpisa**

$f+1$  potpisa i njihovi javni ključevi sabijaju se u JEDAN mali potpis (~100–200 bajtova), bez obzira na broj potpisnika.

# Zašto se ovo radi pre konsenzusa?

## Klasičan pristup

Lider predloži blok → svi ga preuzimaju (2s) → tek onda glasaju (1s).  
Prenos podataka je NA kritičnom putu.

## Autobahn

Podaci se preuzimaju KONSTANTNO u pozadini, i dok se glasa o prethodnom TipCut-u. Glasa se samo o ~1KB hash-eva.

## Zaštita od zloupotrebe

Ako autor CAR-a ne isporuči podatke iako ima PoA — bilo koji od  $f+1$  potpisnika mora da ih retransmituje; ako to ne uradi, njegov potpis je lažan i biva kažnjen (slashing). Nevažće transakcije se ne blokiraju ovde — samo neće uspeti kasnije, tokom izvršenja.

# Pseudokod: diseminacija

```
procedure disseminate(validator_id):  
    while true:  
        batch = mempool.pop_batch()  
  
        prev = my_lane.get_last_car().hash  
        car = new CAR(id=++my_lane.counter, lane_id=validator_id,  
                      prev_hash=prev, batch_hash=hash(batch))  
  
        broadcast(car)  
  
        signatures = []  
        while len(signatures) < f + 1:  
            msg = wait_for_any_signature()  
            if verify_signature(msg) and msg.car_id == car.id:  
                signatures.append(msg.signature)  
  
        car.poa = aggregate_signatures(signatures, f+1)  
        my_lane.append(car)
```

# TipCut

```
struct TipCut
```

```
map<lane_id, CAR> cars
```

Najnoviji CAR (sa PoA) iz svake trake — samo reference (hash-evi), bez ijedne transakcije.

**N lana × 32 bajta ≈ 1KB**

*Umesto da glasaju o gigabajtima transakcija, validatori glasaju o par hiljada bajtova hash-eva.*

Lider gradi predlog:

1

**Pregleda sve trake**

Iz svake trake uzima poslednji CAR koji ima validan PoA.

2

**Formira TipCut**

Spaja te reference u jedan mali paket.

3

**Šalje PREPARE predlog**

Emituje TipCut svim validatorima — počinje glasanje.

## Fast Path — idealni uslovi

Podrazumevani put kada je mreža stabilna i lider ispravan. Samo 1.5 runde, tri koraka poruka.

- 1 Lider šalje PREPARE** TipCut sa referencama na CAR-ove koji imaju PoA.
- 2 Validatori šalju PRE-VOTE** Provere da svaki CAR u TipCut-u ima validan PoA — ne diraju sadržaj transakcija.
- 3 Lider sakupi SVIH n glasova** Agregira ih u Fast Commit QC i odmah ga emituje — Confirm faza se PRESKAČE.

Čim validatori vide Fast Commit QC, TipCut je finalizovan — svi CAR-ovi u njemu i njihovi preci postaju trajno uređeni.

# Slow Path i promena lidera

Kada lider ne dobije SVE glasove na vreme (spor validator, nestabilna mreža) — sistem prelazi na sigurniji, malo duži put.

## Slow Path — 2.5 runde

1. Lider sakupi bar  $2f+1$  pre-glasova → formira P-QC i šalje CONFIRM
2. Validatori vraćaju CONFIRM-ACK
3. Lider sakupi  $2f+1$  potvrda → C-QC → SLOW\_COMMIT i finalizacija

## View Change

1. Ako ni  $2f+1$  glasova ne stigne, validatori šalju TIMEOUT poruke
2. Kad novi lider sakupi  $2f+1$  timeout-a → Timeout Certificate (TC)
3. Svi prelaze u novi view — proces kreće ispočetka sa novim liderom

*Trake i dalje rade tokom View Change-a — novi lider odmah ima šta da stavi u prvi TipCut, pa oporavak nema hangover.*

# Pseudokod: konsenzus - lider

```
procedure leader_propose(lider_id):  
    tipcut = build_tipcut()  
    broadcast(PREPARE, tipcut)  
  
    votes = collect_votes(timer)  
  
    if len(votes) == N:  
        broadcast(FAST_COMMIT, aggregate(votes))  
        return  
  
    if len(votes) >= 2*f + 1:  
        pqc = aggregate(votes)  
        broadcast(CONFIRM, pqc)  
        acks = collect_confirm_acks(timer)  
        if len(acks) >= 2*f + 1:  
            broadcast(SLOW_COMMIT, aggregate(acks))  
            return  
  
    initiate_view_change()
```



# Pseudokod: konsenzus - validatori

```
procedure validator_consensus():  
    while true:  
        msg = wait_for_any_message()  
  
        if msg.type == PREPARE:  
            tipcut = msg.tipcut  
  
            if verify_all_poa(tipcut.cars):  
                send_to(msg.lider, PRE_VOTE, {tipcut_hash: hash(tipcut), signature: sign(hash(tipcut))})  
  
        elif msg.type == FAST_COMMIT:  
            if verify_qc(msg.fast_qc, N):  
                finalize(msg.tipcut)  
  
        elif msg.type == CONFIRM:  
            if verify_qc(msg.pqc, 2*f+1):  
                send_to(msg.lider, CONFIRM_ACK, {pqc_hash: hash(msg.pqc), signature: sign(hash(msg.pqc))})  
  
        elif msg.type == SLOW_COMMIT:  
            if verify_qc(msg.commit_qc, 2*f+1):  
                finalize(msg.tipcut)  
  
        elif timeout_reached():  
            send_to_all(TIMEOUT, {view: current_view + 1})
```

# Finalizacija i izvršenje

1

## Proširi TipCut kroz lance

Za svaku traku u TipCut-u, prati prev\_hash unazad do CAR-a koji je već izvršen — tako se pokupe i svi PRECI, ne samo poslednji CAR.

2

## Deterministički sortiraj

Primeni fiksno pravilo (npr. redom po lanama, pa po visini) — svi validatori dobijaju IDENTIČAN redosled iz istog TipCut-a.

3

## Izvrši transakcije

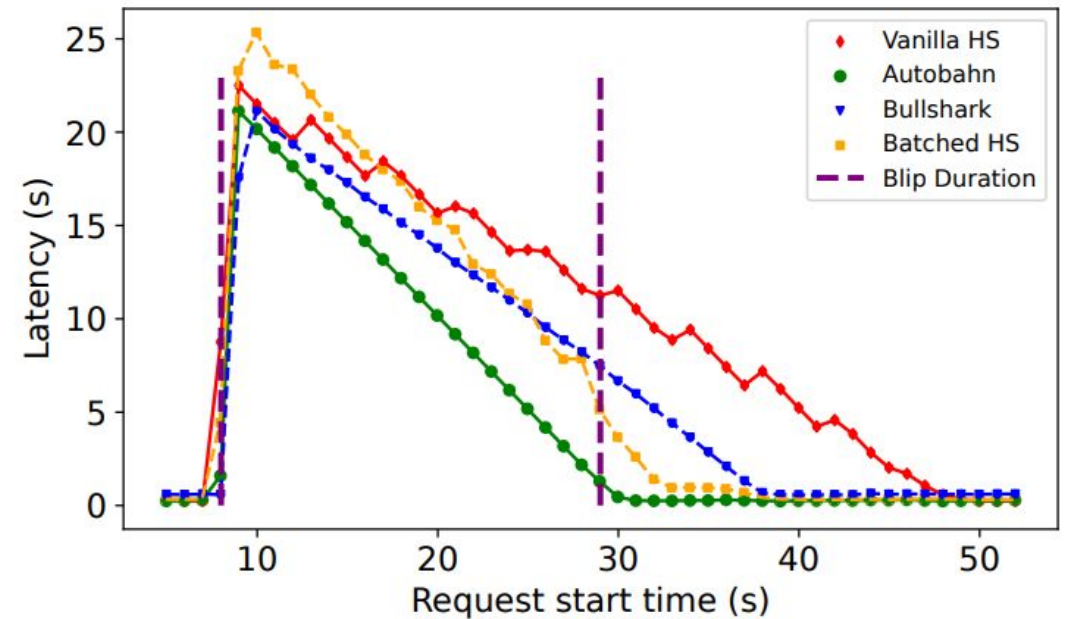
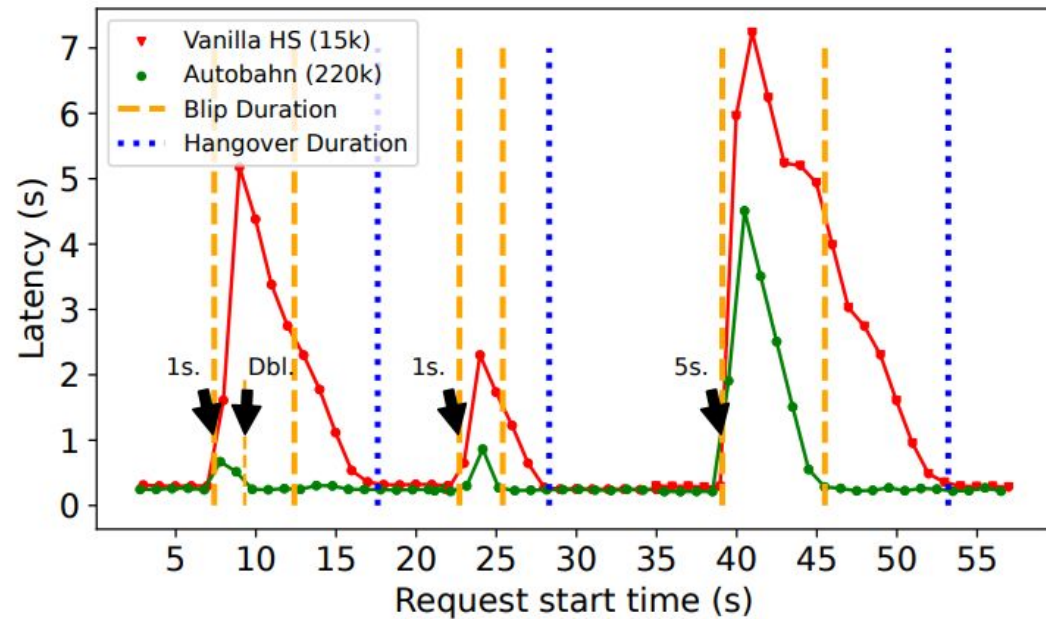
Tek sada se proverava stanje (balans, uslovi) — nevažeće transakcije jednostavno budu odbačene kao „failed“, lanac se ne zaustavlja.

*Ovde se prvi put u celom procesu proverava logika transakcija — dosad je provera bila samo sintaksna i dostupnosna.*

# Pseudokod: finalizacija

```
procedure finalize(tipcut):  
    all_cars = []  
  
    for each lane in tipcut.cars:  
        car = tipcut.cars[lane.id]  
        while car is not null:  
            all_cars.append(car)  
            car = get_car_by_hash(car.prev_hash)  
  
    ordered_cars = deterministic_sort(all_cars)  
  
    for each car in ordered_cars:  
        for each tx in get_transactions(car.batch_hash):  
            execute(tx)  
  
    commit_state()
```

# Rezultati



# Primena u praksi

## Sei Giga

Autobahn se koristi kao mehanizam konsenzusa (Proof-of-Stake) u blockchain sistemu Sei Giga, dizajniranom da bude performantan u modelu delimične sinhronosti, slično HotStuff-u.

### Otvoreni izvorni kod

Referentna implementacija dostupna je javno (autobahn-artifact na GitHub-u), sa rezultatima eksperimenata throughput-a, latencije i skalabilnosti.



### Zašto je ovo bitno

- Pokazuje da je pristup primenjiv van akademskog rada
- Otvara vrata bržim i stabilnijim blockchain mrežama